



第七章 输入输出

主要内容



华中科技大学

- ◆ 7.1. 输入输出子系统
- ◆ 7.2. I/O端口和I/O指令
- ◆ 7.3. 数据传送方式





7.1.输入输出子系统

I/O子系统是计算机的重要部分。编写用户程序、使用操作系统时，会涉及外部设备I/O。高级语言用编译系统提供的I/O库函数实现外设I/O，通过陷阱指令转化为操作系统内核执行。



7.1.输入输出子系统

I/O子系统解决信息输入输出问题。各种信息通过外设输入计算机，计算机结果通过外设输出用户。高级语言有I/O机制，如C语言中的printf()、scanf()。用户用I/O函数或操作符提出I/O请求，设备响应完成请求，需要I/O软件和硬件协调。



7.1.输入输出子系统

I/O子系统包括I/O软件和I/O硬件。

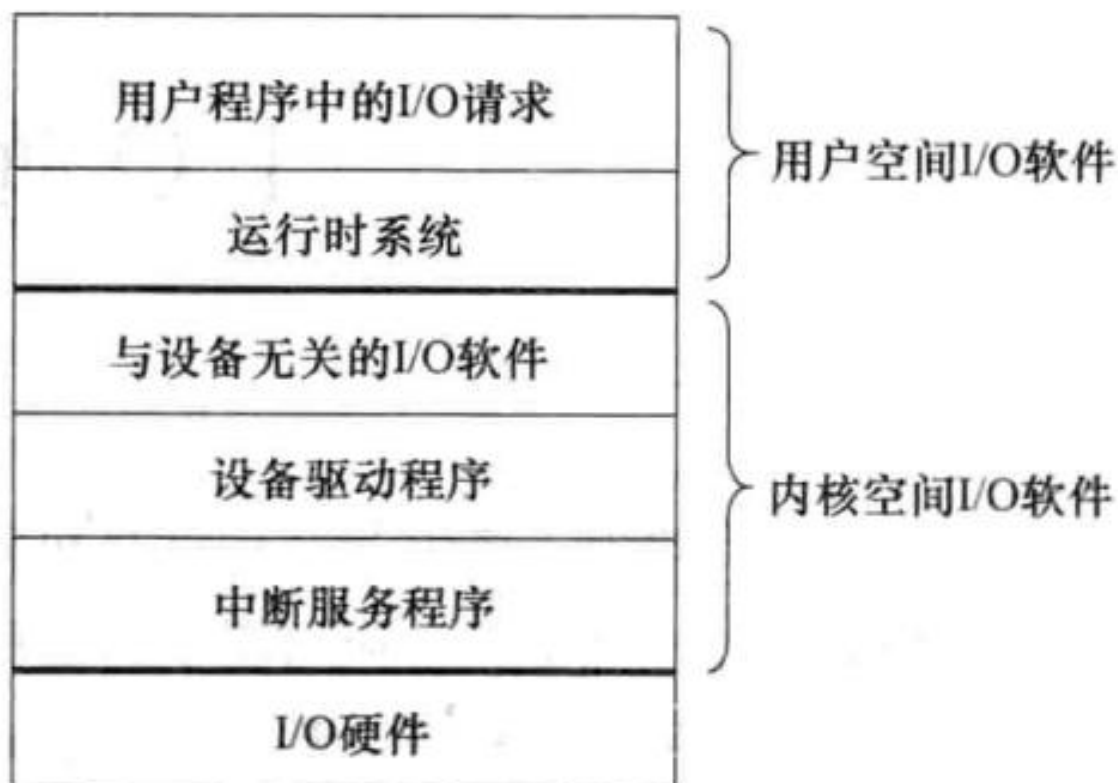
I/O软件有用户空间I/O软件和内核空间I/O软件。

I/O硬件受内核空间I/O软件控制。



7.1.输入输出子系统

I/O子系统层次结构:





7.1.输入输出子系统

操作系统在I/O子系统扮演重要角色，实现了外部设备I/O的共享性、复杂性和异步性。用户程序通过I/O函数或操作符请求I/O操作，最终通过内核的系统调用实现

。



7.1.输入输出子系统

输入输出子系统中的概念:

- C语言I/O库函数: 如printf()、scanf()
- 操作系统系统调用的封装函数: 如read()、write()
- 系统调用服务例程: 如system_call()、sys_write()。
- 陷阱指令: 如int n、syscall n
- 文件: 对设备的封装, 如磁盘文件、键盘、打印机
- 设备驱动程序: 设备商自己提供的功能程序, 与设备有关的功能模块都在此。

设备无关I/O软件层



华中科技大学

◆ 设备驱动程序统一接口

- 操作系统为**所有外设的设备驱动程序规定一个统一接口**，这样，新设备的驱动程序只要按统一接口规范来编制，就可在**不修改操作系统**的情况下，添加新设备驱动程序并使用新的外设进行I/O。
- **所有设备都抽象成文件**，设备名和文件名在形式上没有差别，**设备和文件具有统一的接口**，不同设备名和文件名被映射到对应设备驱动程序。

◆ 缓冲处理

- 每个设备的I/O都需**使用内核缓冲区**，因而缓冲区的申请和管理等处理是所有设备公共的，可包含在与设备无关的I/O软件部分

◆ 错误报告

- I/O操作在内核态执行时所发生的错误信息，都通过与设备无关的I/O软件返回给用户进程，也即：**错误处理框架与设备无关**。
- **直接返回编程等错误，无需设备驱动程序处理**，如，请求了不可能的I/O操作；写信息到一个输入设备或从一个输出设备读信息；指定了一个无效缓冲区地址或者参数；指定了不存在的设备等。
- 有些错误由设备驱动程序检测出来并处理，若驱动程序无法处理，则将错误信息返回给设备无关I/O软件，**再由设备无关I/O软件返回给用户进程**，如写**一个**已被破坏的磁盘扇区；打印机缺纸；读一个已关闭的设备等。



◆ 打开与关闭文件

- 对设备或文件进行打开或关闭等I/O函数所对应的系统调用，并不涉及具体的I/O操作，只要直接对主存中的一些数据结构进行修改即可，这部分工作也由设备无关软件来处理。

◆ 逻辑块大小处理

- 为了为所有的块设备和所有的字符设备分别提供一个统一的抽象视图，以隐藏不同块设备或不同字符设备之间的差异，与设备无关的I/O软件为所有块设备或所有字符设备设置统一的逻辑块大小。
- 对于块设备，不管磁盘扇区和光盘扇区有多大，所有逻辑数据块的大小相同，这样，高层I/O软件就只需处理简化的抽象设备，从而在高层软件中简化了数据定位等处理。

设备驱动程序



华中科技大学

- ◆ 每个外设具体的I/O操作需通过执行设备驱动程序来完成
- ◆ 外设种类繁多、其控制接口不一，导致不同外设的**设备驱动程序千差万别**，因而设备驱动程序与设备相关
- ◆ 每个外设或每类外设都有一个**设备控制器**，其中包含各种**I/O端口**。CPU通过执行设备驱动程序中的**I/O指令**访问各种I/O端口
- ◆ 设备所采用的I/O控制方式不同，驱动程序的实现方式也不同
 - **程序直接控制**：驱动程序完成用户程序的I/O请求后才结束。这种情况下，用户进程在I/O过程中不会被阻塞，内核空间的I/O软件一直代表用户进程在内核态进行I/O处理。（干等！）
 - **中断控制**：驱动程序启动第一次I/O操作后，将调出其他进程执行，而当前用户进程被阻塞。在CPU执行其他进程的同时，外设进行I/O操作，此时，CPU和外设并行工作。外设完成I/O时，向CPU发中断请求，然后CPU调出相应中断服务程序执行。在中断服务程序中再次启动I/O操作。
 - **DMA控制**：驱动程序对DMA控制器初始化后，便发送“启动DMA传送”命令，外设开始进行I/O操作并在外设和主存间传送数据。同时CPU执行处理器调度程序，转其他进程执行，当前用户进程被阻塞。DMA控制器完成所有I/O任务后，向CPU发送一个“DMA完成”中断请求信号。





7.1.输入输出子系统

设备驱动与中断处理的关系：

中断处理是一套调用机制，也是一种CPU与外设交流的机制。由于外设种类众多，CPU与外设交流功能的实现，分为统一的抽象部分，和基于设备不同的具体部分。抽象部分由操作系统实现，具体部分由设备驱动实现。两者都利用了中断处理机制。



7.1.输入输出子系统

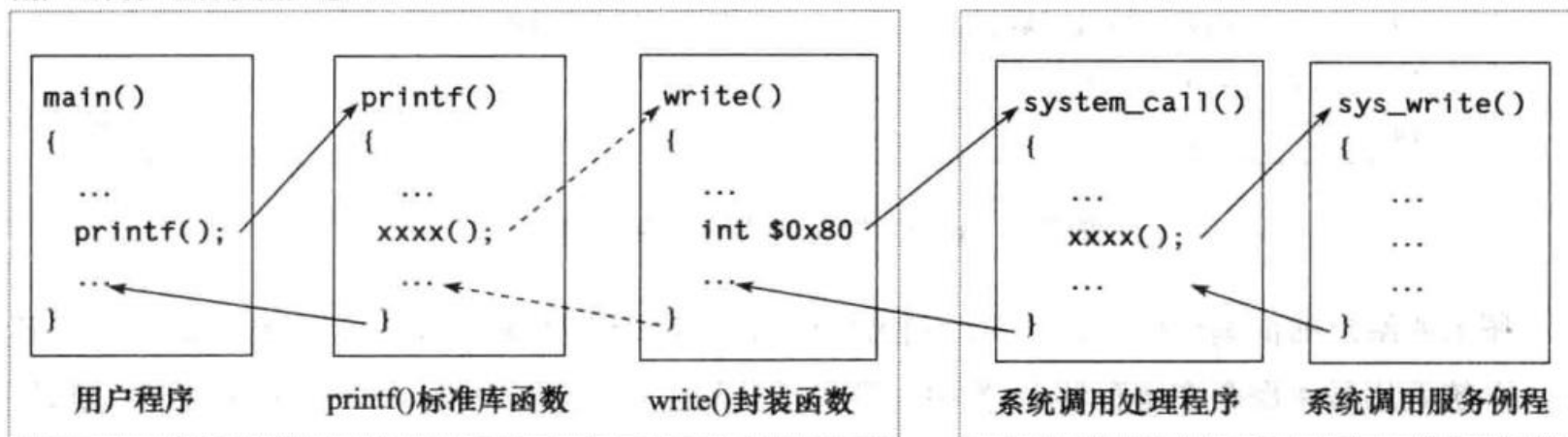
例如设备驱动提供了设备的具体的中断处理程序，并通过操作系统注册了这些中断处理程序。抽象的系统功能调用，会将请求传递到这些具体的中断处理程序上来。操作系统和设备驱动配合，完成具体的外部设备的访问。

7.1.输入输出子系统

例：在Linux中printf()的调用过程

用户空间，运行在用户态

内核空间，运行在内核态



主要内容



华中科技大学

- ◆ 7.1. 输入输出子系统
- ◆ 7.2. I/O端口和I/O指令
- ◆ 7.3. 数据传送方式





7.2.I/O端口和I/O指令

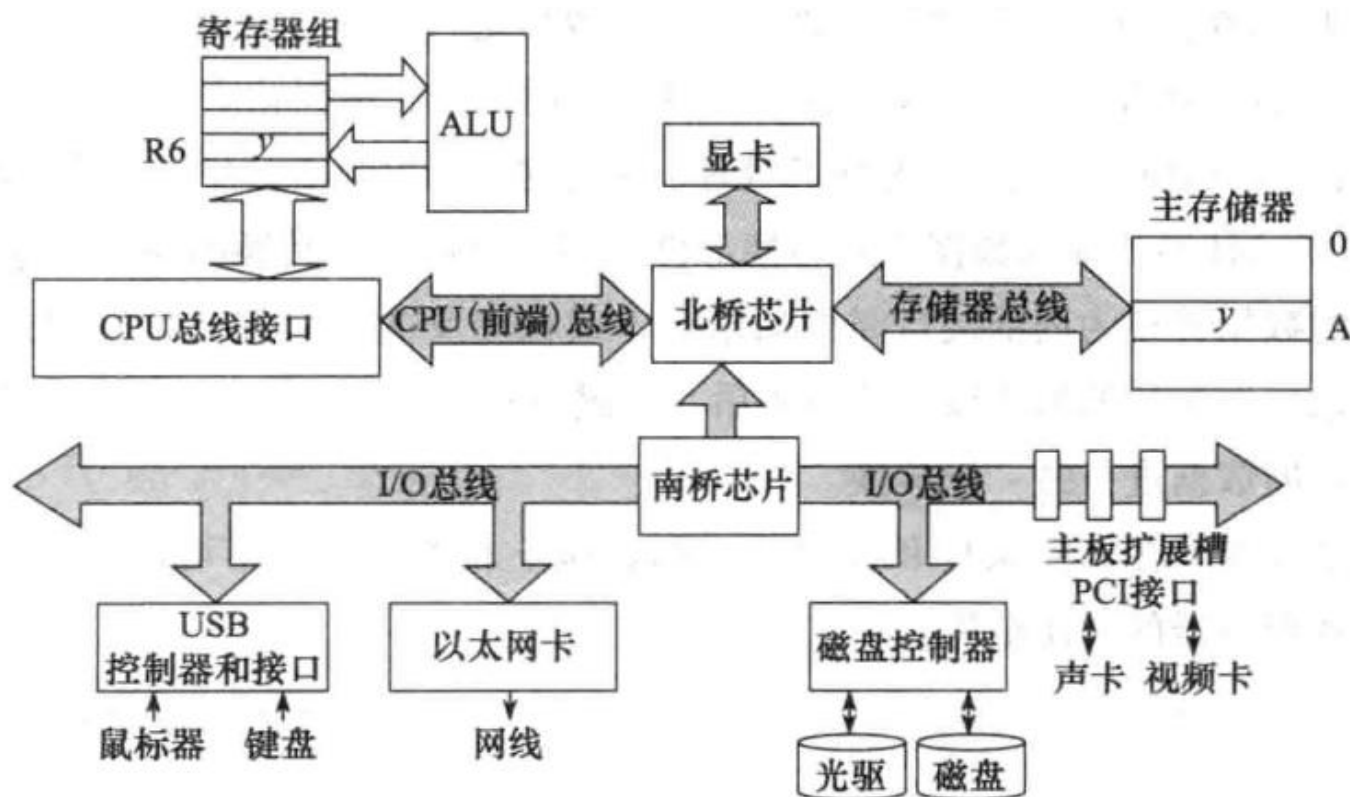
I/O硬件通常由机械部分和电子部分组成。机械部分是I/O设备本身，而电子部分则称为设备控制器或I/O适配器。

I/O总线用来在系统中连接多个设备控制器和总线桥接器，物理上它是主板上的一些I/O扩展槽，通常是标准总线。目前典型的I/O总线标准是PCI-Express。



7.2.1/O端口和I/O指令

外设、设备控制器和CPU、主存的连接





7.2.I/O端口和I/O指令

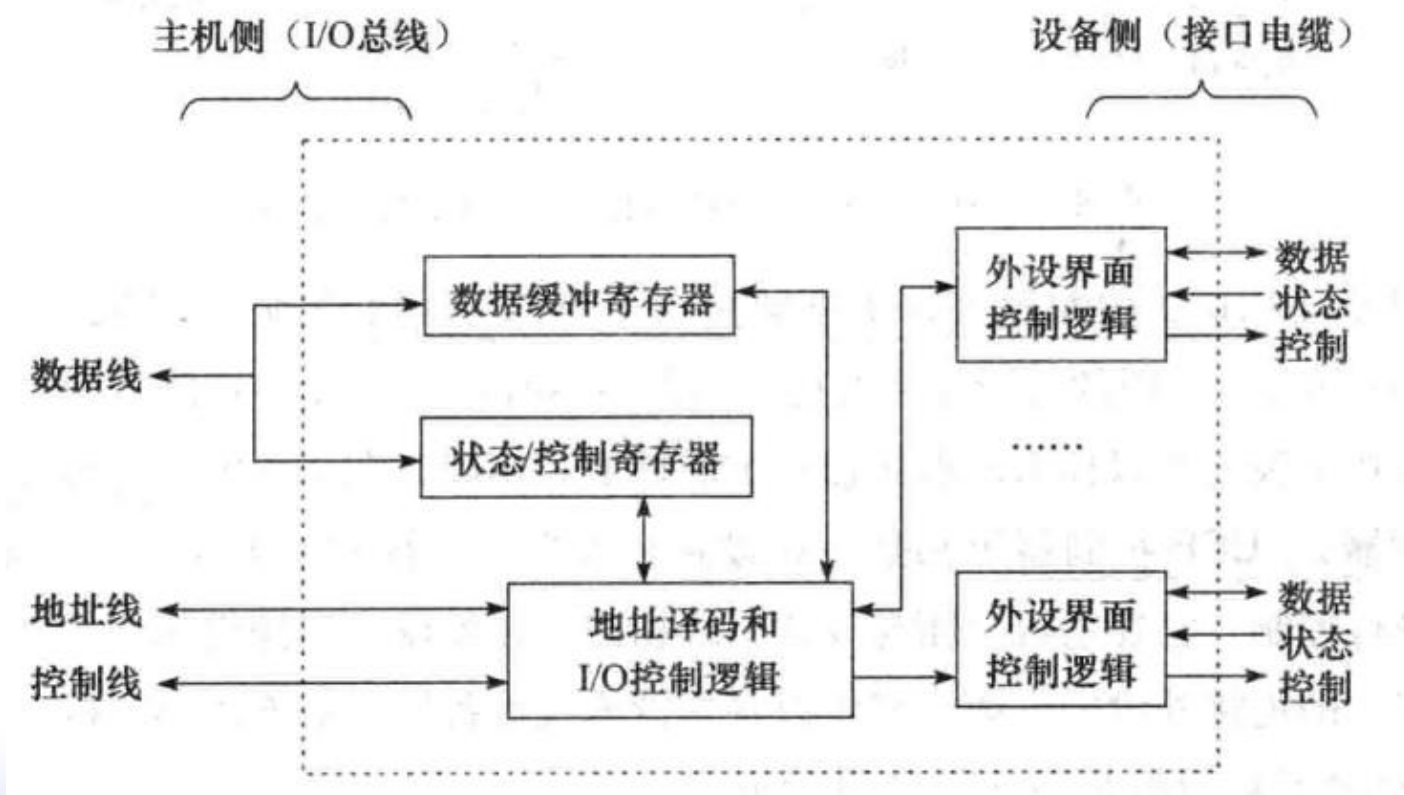
设备控制器中包含数据缓冲寄存器、状态/控制寄存器等多个不同的寄存器，用于存放外设与主机交换的数据信息、控制信息和状态信息。

通常把设备控制器中的数据缓冲寄存器、状态/控制寄存器等统称为I/O端口(I/O port)。数据缓冲寄存器简称为数据端口，状态/控制寄存器简称为状态/控制端口。



7.2.I/O端口和I/O指令

设备控制器的通用结构





7.2.I/O端口和I/O指令

为了便于CPU对外设的快速选择和对I/O端口的寻址，必须对I/O端口进行编址，所有I/O端口编号组成的空间称为I/O地址空间。I/O端口的编址方式有两种：统一编址方式和独立编址方式。

独立编址方式对所有的I/O端口单独进行编号，使它们成为一个独立的I/O地址空间。这种情况下，指令系统中需要有专门的I/O指令来访问I/O端口，I/O指令中地址码部分给出I/O端口号。

IA-32中，允许设置64K个8位端口或32K个16位端口





华中科技大学

7.2.I/O端口和I/O指令

例：

40H ~ 43H时钟/定时器，60H ~ 63H为8255通讯芯片的接口。





7.2.I/O端口和I/O指令

在底层I/O软件中，可以将控制命令送到控制寄存器来启动外设工作；可以读取状态寄存器来了解外设和设备控制器的状态；可以通过直接访问数据缓冲寄存器来进行数据的输入和输出。这些操作是通过相应的指令来完成的，通常把这类指令称为I/O指令。这些I/O指令只能在操作系统内核的底层I/O软件中使用，是一种特权指令。



1. 输入指令IN

功能：从外设寄存器取信息送入AX或AL。

形式：IN \$PORT, %AL,

功能：(PORT) → AL

例：

in \$PORT, %ax # (PORT+1, PORT) → AX

in %dx, %al # ([DX]) → AL

in %dx, %ax # ([DX]) → AX

注：由AL或AX决定取几个端口的内容。



2. 输出指令OUT

功能： 将AX或AL内容送到外设寄存器中。

形式： OUT %AL, \$PORT

功能： (AL) → PORT

例：

out %ax, \$PORT # AX→PORT (2个8位口)

out %al, %dx # AL→[DX]

out %ax, %dx # AX→[DX]



7.2.I/O端口和I/O指令

例：

`in $0x40, %al`

`out %al, $0x80`

`mov $0x379, %dx`

`in %dx, %al`

注：端口地址 > 255，就放到DX中

7.2.I/O端口和I/O指令



華中科技大學

INS: 串輸入指令

OUTS: 串輸出指令





7.3.数据传送方式

1.无条件传送方式

特点：不查询外设状态，直接用IN和OUT指令传送信息。
采用这种方式，必须保证外设与CPU在传送数据的过程中，具有相同速度（同步）

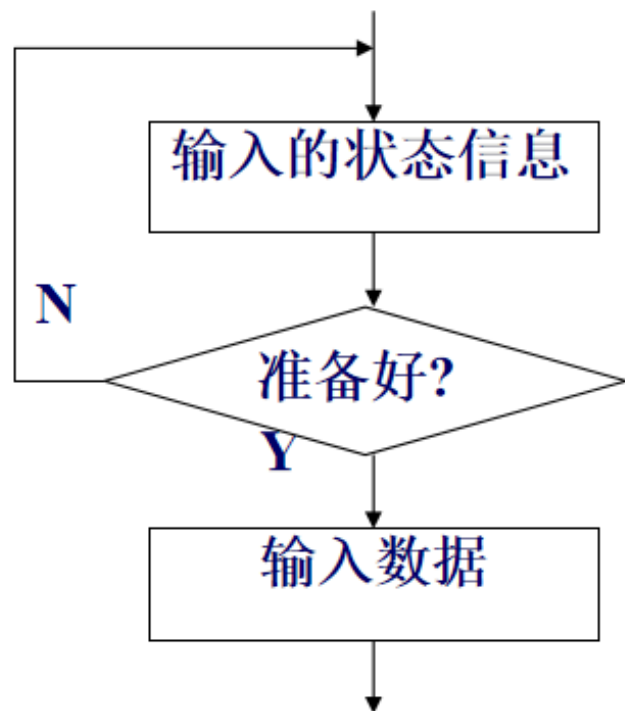


2. 查询传送方式

特点：查询外设状态，条件满足时传送。适合于低速外设与CPU传送信息。

① 输入

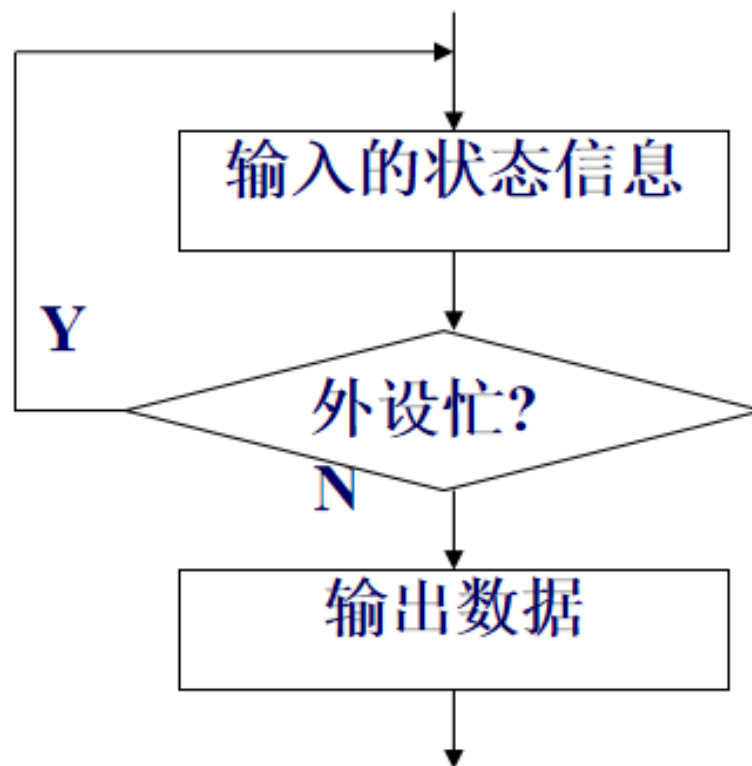
输入之前，查询数据是否准备好，若准备好则输入，否则等待。



2. 查询传送方式

② 输出

输出之前，要查询外设是否“忙”，若“忙”则等待，否则输出数据。





3. 中断传送方式

为了解决查询方式下，CPU等待外设，浪费大量CPU资源的问题，提出了中断传送方式。

CPU启动I/O设备后，不再查询外设I/O工作的状态，而是去执行另外的程序。当外设做好了进行I/O操作的准备后，就向CPU发出中断申请，这时，CPU就暂时中断现行程序的执行，而转去用很少的时间完成此次I/O操作，然后，CPU又恢复执行原来被中断的程序。

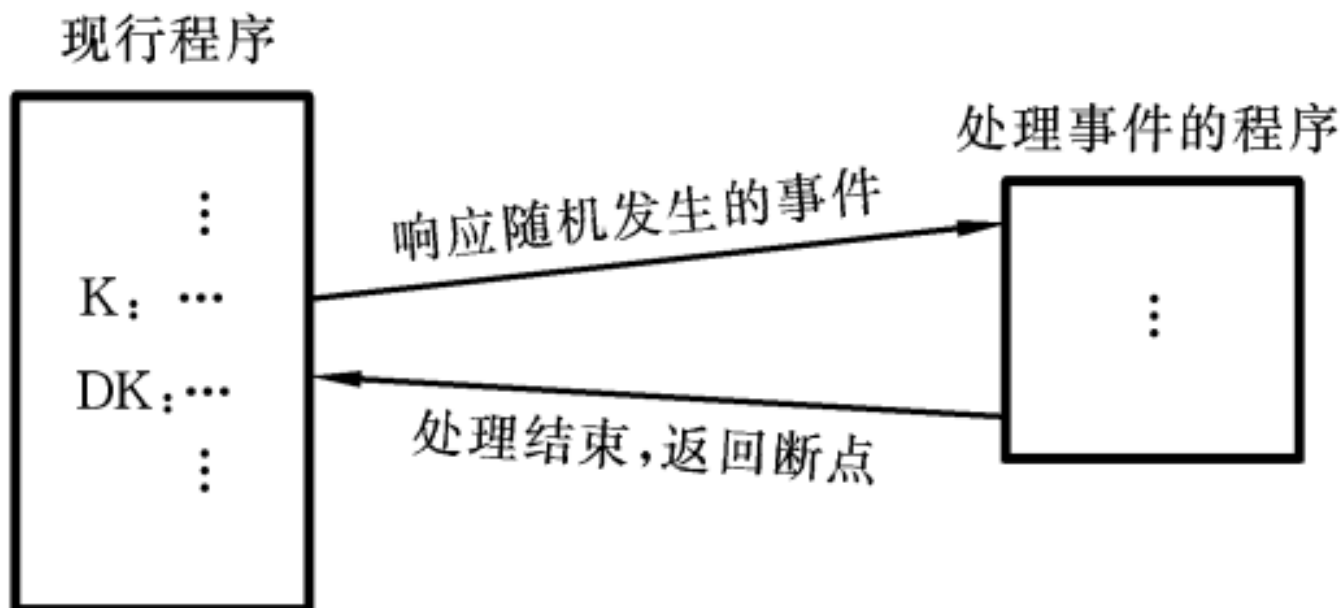


3. 中断传送方式



华中科技大学

中断执行方式



3. 中断传送方式



华中科技大学

有如下优点:

- ① 提高CPU效率
- ② 可以处理突发事件
- ③ 提高了计算机工作的灵活性



4. 直接存储器（又称DMA）传送方式



华中科技大学

适合于高速外存与主机的数据传送。

在端口上，采用通道技术实现。在计算机准备好接收，外设准备好传送数据时，外设向CPU发一个请求DMA传送信号，此时CPU让出总线控制权，使外设与主存在短时间内成批传送数据。

